Arduino Coding and Logic Implementation for Airbag ECU System

## 01. Purpose

The purpose of the software implementation is to demonstrate how an Airbag ECU can read sensor inputs, process crash conditions, control airbag outputs, monitor faults, and communicate safety-relevant data through CAN. This part follows the project instruction that the software must implement sensor reading, output setting, output readback, memory-safe logic, CAN sending, CAN reading, and test routines.

For this university project, the Arduino system is used as a prototype model only. It does not represent a real automotive-certified airbag controller. The goal is to show the functional safety logic in software.

ISO 26262 is considered because it addresses hazards caused by malfunctioning behaviour of electrical and electronic safety-related systems in road vehicles.

## 2. Software Requirement

| Requirement ID | Software Requirement | Purpose |
|---|---|---|
| SWR-01 | The software shall read acceleration sensor input. | Crash detection |
| SWR-02 | The software shall read gyroscope angular sensor input. | Rollover/angular motion detection |
| SWR-03 | The software shall read pressure sensor input. | Side-impact detection |
| SWR-04 | The software shall read passenger seat digital input. | Passenger airbag suppression |
| SWR-05 | The software shall activate driver airbag output when valid crash is detected. | Driver protection |
| SWR-06 | The software shall activate passenger airbag only if no baby seat is detected. | Passenger safety |
| SWR-07 | The software shall activate warning lamp during system fault. | Fault indication |
| SWR-08 | The software shall send crash/fault status through CAN. | ECU communication |
| SWR-09 | The software shall read CAN messages for diagnostic or reset command. | Communication handling |
| SWR-10 | The software shall read back output status after activation. | Output verification |
| SWR-11 | The software shall enter safe state if fault is detected. | Prevent unintended deployment |
| SWR-12 | The software shall support test routines for module and integration testing. | Verification |

## 3. Selected Platform

Arduino Due is suitable because it is based on a 32-bit ARM Cortex-M3 microcontroller and has 54 digital input/output pins, 12 analog inputs, and an 84 MHz clock. Arduino Due is also suitable for CAN-based demonstrations because CAN communication libraries exist for the Arduino Due. The 'due_can' library supports CAN bus communication and interrupt-driven communication on Arduino Due.

For simpler demonstration, Arduino Uno can also be used in Tinkercad, but for matching the professor's instruction, the report should write Arduino Due as the intended board.

## 4. Input and Output Definition

### 4.1 Input Signals

| Input | Arduino Pin | Type | Description |
|---|---|---|---|
| Acceleration sensor | A0 | Analog | Detects strong frontal acceleration |
| Gyroscope sensor | A1 | Analog | Detects angular/rollover movement |
| Pressure sensor | A2 | Analog | Detects side impact pressure |
| Passenger/baby seat switch | D2 | Digital | Disables passenger airbag |
| CAN receive | CAN RX | Digital/CAN | Receives ECU command/status |

### 4.2 Output Signals

| Output | Arduino Pin | Type | Description |
|---|---|---|---|
| Driver airbag indicator | D8 | Digital | Simulated driver airbag deployment |
| Passenger airbag indicator | D9 | Digital | Simulated passenger airbag deployment |
| Warning lamp | D13 | Digital | Airbag fault warning lamp |
| CAN transmit | CAN TX | CAN | Sends crash/fault status |

## 5. Software Logic Description

As the software follows a cyclic embedded control system, the main loop performs the following tasks. The important safety idea is that the airbag must not deploy only because one sensor value is high, rather the software requires a valid crash condition and a safing condition before deployment which reduces the risk of unintended deployment.

- Read all sensor inputs.
- Convert sensor values into logical crash conditions.
- Check passenger/baby seat input.
- Run safing validation.
- Detect faults.
- Decide whether airbag deployment is allowed.
- Activate correct outputs.
- Read back output status.
- Send CAN diagnostic frame.
- Repeat continuously.

## 6. Crash Detection Logic

| Sensor | Normal Range | Crash Threshold |
|---|---|---|
| Acceleration sensor | 0–699 | ≥700 |
| Gyroscope sensor | 0–599 | ≥600 |
| Pressure sensor | 0–649 | ≥650 |

- The crash detection condition is:

Frontal crash = acceleration sensor ≥ 700
Side crash = pressure sensor ≥ 650
Safing condition = gyroscope sensor ≥ 600 OR acceleration sensor ≥ 700

- Deployment is allowed when:

Crash Detected = TRUE
AND Safing Condition = TRUE
AND System Fault = FALSE

- Passenger airbag deployment requirement (additionally):

Baby Seat Detected = FALSE

## 7. Safe State Logic

The system enters safe mode if it satisfies the following conditions:

- Sensor value is outside valid range
- Output readback does not match the command
- CAN error is detected
- Invalid passenger seat signal is detected

So the safe state can be defined as-

Driver airbag output = OFF
Passenger airbag output = OFF
Warning lamp = ON
CAN fault message = SENT

## 8. CAN Frame Design

CAN allows the airbag to send high priority crash or fault information to the main vehicle ECU. Arduino documentation describes CAN as a vehicle bus standard that allows microcontrollers and devices to communicate with each other without a host computer.

| Field | Value |
|---|---|
| CAN ID | 0x120 |
| DLC | 8 bytes |
| Byte 0 | System status |
| Byte 1 | Acceleration value |
| Byte 2 | Gyroscope value |
| Byte 3 | Pressure value |
| Byte 4 | Passenger/baby seat status |
| Byte 5 | Deployment status |
| Byte 6 | Fault code |
| Byte 7 | Simple checksum |

## 9. Arduino Code for Airbag ECU Prototype

```
const int accSensor = A0;
const int gyroSensor = A1;
const int pressureSensor = A2;

const int babySeatPin = 2;

const int driverAirbag = 8;
const int passengerAirbag = 9;
const int warningLamp = 13;

const int ACC_THRESHOLD = 700;
const int GYRO_THRESHOLD = 600;
const int PRESSURE_THRESHOLD = 650;

void setup() {

  Serial.begin(9600);

  pinMode(driverAirbag, OUTPUT);
  pinMode(passengerAirbag, OUTPUT);
  pinMode(warningLamp, OUTPUT);

  pinMode(babySeatPin, INPUT_PULLUP);

  digitalWrite(driverAirbag, LOW);
  digitalWrite(passengerAirbag, LOW);
  digitalWrite(warningLamp, LOW);
```

```
  Serial.println("Airbag ECU Simulation Started");
}

void loop() {

  int accValue = analogRead(accSensor);
  int gyroValue = analogRead(gyroSensor);
  int pressureValue = analogRead(pressureSensor);

  bool babySeatDetected =
  (digitalRead(babySeatPin) == LOW);

  bool frontalCrash =
    accValue >= ACC_THRESHOLD;

  bool sideCrash =
    pressureValue >= PRESSURE_THRESHOLD;

  bool safingCondition =
    gyroValue >= GYRO_THRESHOLD;

  // Reset outputs
  digitalWrite(driverAirbag, LOW);
  digitalWrite(passengerAirbag, LOW);
  digitalWrite(warningLamp, LOW);

  Serial.println("------------------------------");

  Serial.print("ACC=");
  Serial.print(accValue);

  Serial.print(" | GYRO=");
  Serial.print(gyroValue);

  Serial.print(" | PRESSURE=");
  Serial.print(pressureValue);

  Serial.print(" | BABY=");
  Serial.println(babySeatDetected ? "YES" : "NO");

  // Crash condition
  if ((frontalCrash || sideCrash) && safingCondition) {

    digitalWrite(driverAirbag, HIGH);

    if (babySeatDetected) {

      digitalWrite(passengerAirbag, LOW);
```

```
    Serial.println("SCENARIO: VALID CRASH + BABY SEAT");
    Serial.println("Driver Airbag ON");
    Serial.println("Passenger Airbag SUPPRESSED");
   }
   else {

    digitalWrite(passengerAirbag, HIGH);

    Serial.println("SCENARIO: VALID CRASH");
    Serial.println("Driver Airbag ON");
    Serial.println("Passenger Airbag ON");
   }
  }
  else {

   Serial.println("SCENARIO: NORMAL / NO VALID CRASH");
   Serial.println("Driver Airbag OFF");
   Serial.println("Passenger Airbag OFF");

   // Warning / Fault logic
   if(accValue > 950 ||
     gyroValue > 950 ||
     pressureValue > 950) {

    digitalWrite(warningLamp, HIGH);

    Serial.println("WARNING: SENSOR VALUE TOO HIGH");
   }
  }

  delay(1000);
}
```

## 10. Code Explanation

- **Setup ()** : It initializes the ECU prototype which sets the sensor pins, output pins and warning lamp and serial communication
- **Loop() :** Works as the cyclic ECU task which continuously reads the sensor inputs, evaluates the crash logic and checks the safety conditions and send the diagnostic information to make the decision about the deployment.
- **checkSensorValidity():** This checks weather the sensor values are inside valid ADC range. If invalid data is detected then the software goes into safe state.
- **evaluateCrashLogic() :** This converts the raw sensor readings into crash condition.

- **evaluateDeplymentDecision() :** This checks whether the crash and safing conditions are both valid before allowing deployment.
- **deployAirbag() :** This activates the outputs. The driver airbag is activated during valid crash. The passenger airbag is activated only when the baby seat is not detected.
- **verifyOutputState() :** It performs output readback verification.
- **enterSafeState():** Disables deployment output and turn on the warning lamp.
- **sendDiagnosticFrame() :** This creates an 8-byte diagnostic frame representing CAN communication. In real Arduino Due implementation, which can be connected to the 'due_can' library.

**11. Test Routine Plan**

| Test ID | Test Condition | Input Values | Expected Output |
|---|---|---|---|
| ST-01 | Normal driving | ACC=300, GYRO=200, PRESSURE=250 | No airbag, warning lamp OFF |
| ST-02 | Frontal crash | ACC=800, GYRO=650, PRESSURE=300 | Driver + passenger airbag ON |
| ST-03 | Side crash | ACC=400, GYRO=650, PRESSURE=800 | Driver + passenger airbag ON |
| ST-04 | Baby seat detected | ACC=800, GYRO=650, baby seat=ON | Driver ON, passenger OFF |
| ST-05 | No safing condition | ACC=500, GYRO=300, PRESSURE=800 | No deployment |
| ST-06 | Output readback fault | Command HIGH but read LOW | Safe state |
| ST-07 | Fault condition | Invalid sensor/fault flag | Warning lamp ON |
| ST-08 | CAN frame check | Any valid input | 8-byte frame printed |

**12. Coverage Plan**

| Coverage Type | How to Achieve |
|---|---|
| Statement coverage | Run tests so every function executes at least once |
| Branch coverage | Test both TRUE and FALSE conditions of every 'if' statement |
| Path coverage | Test normal path, crash path, baby seat path, and fault path |
| Integration coverage | Test sensor input → logic → output → diagnostic frame |
| System verification | Run full scenario on Arduino/test board |

**Project Link:** https://wokwi.com/projects/463574030977197057

**Figures:**

**Fig 01: Arduino Connections**

Fig 02: Normal State

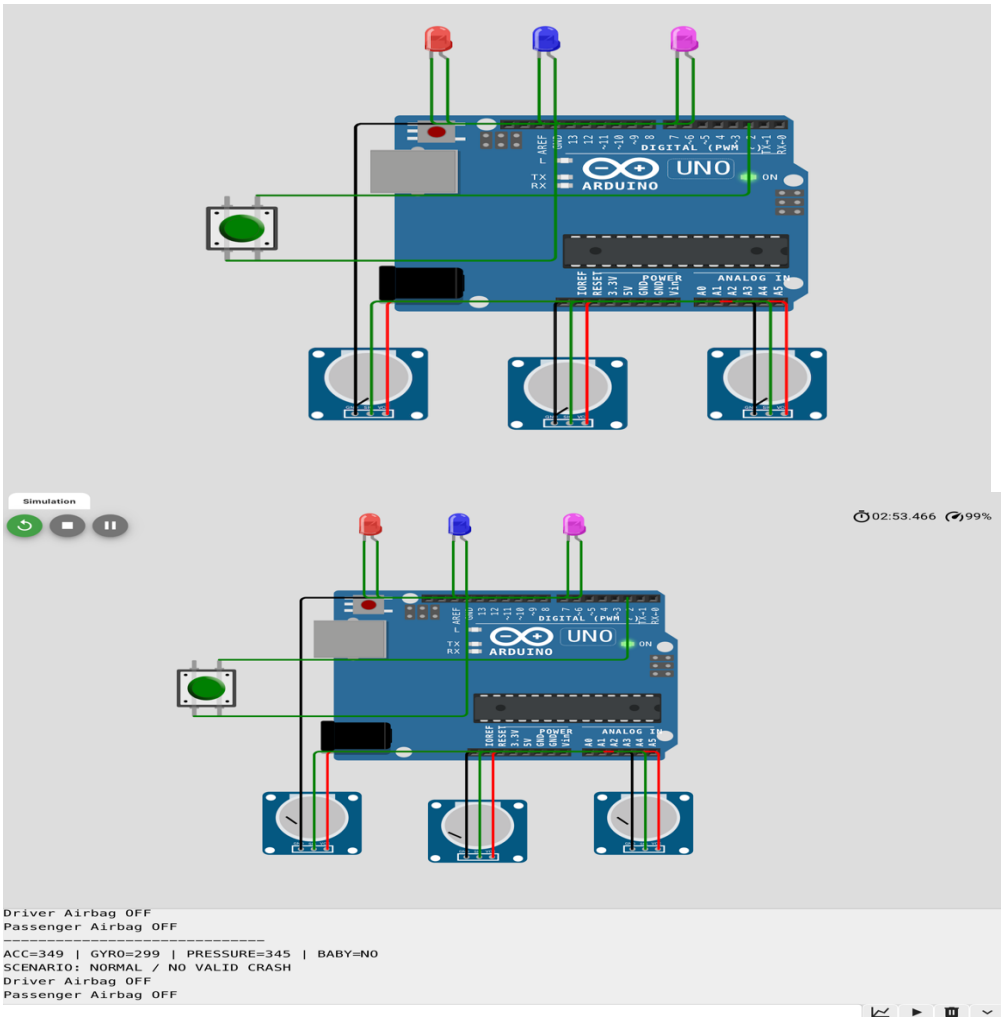



Fig 03: Signal with defective Airbag

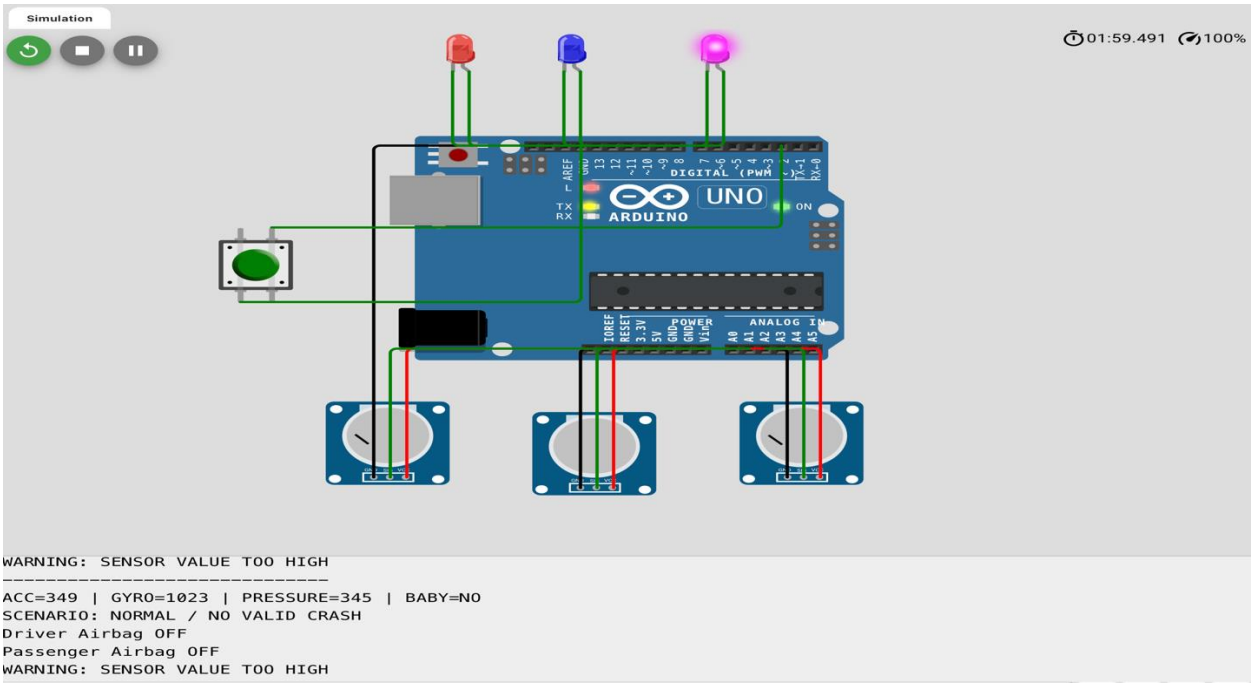



Fig 04: Baby Seat Detected

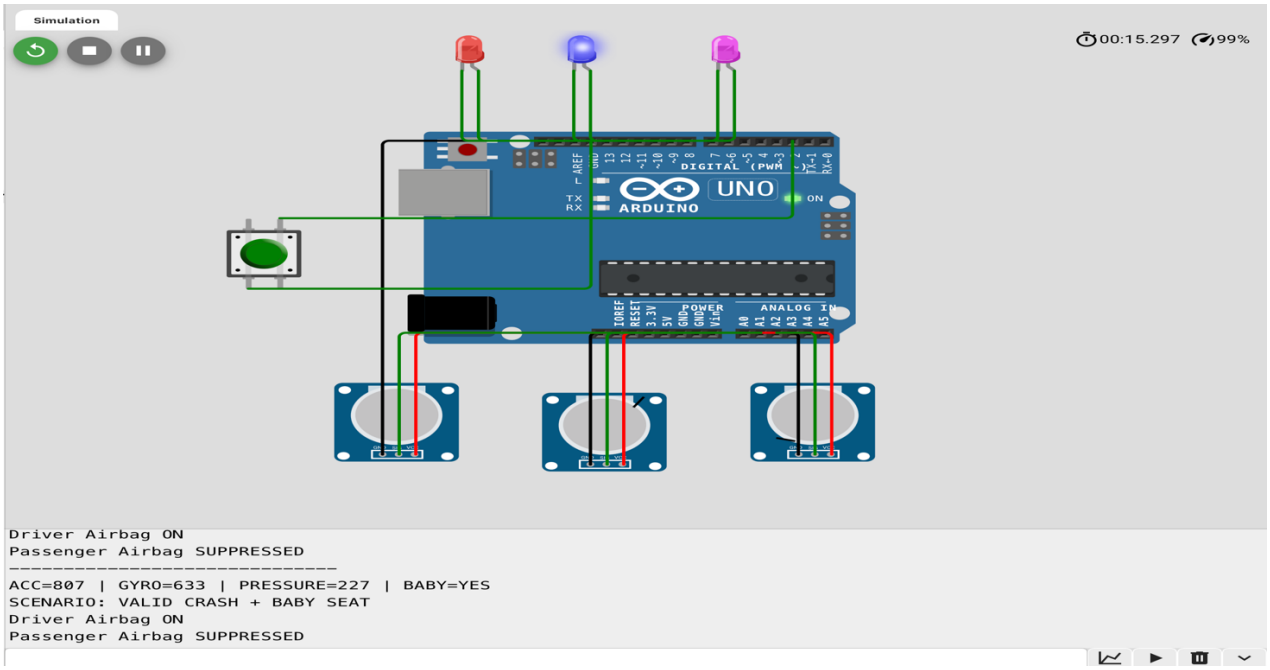
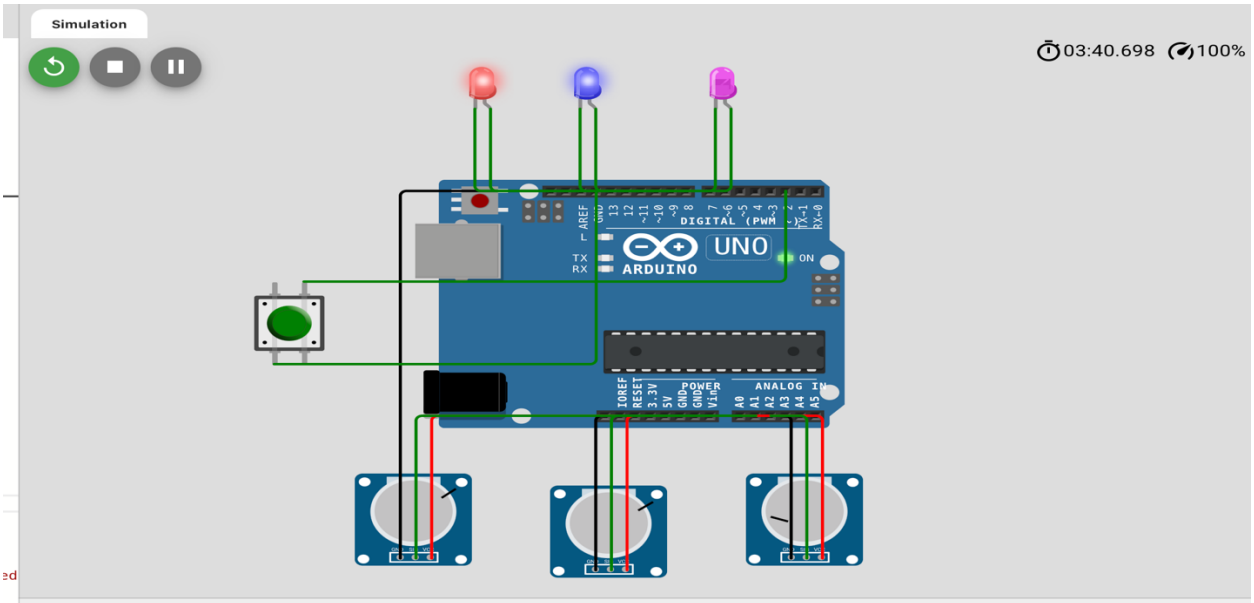



Fig 5: Frontal and Side Accident detected




```
------------------------------
ACC=705 | GYRO=716 | PRESSURE=242 | BABY=NO
SCENARIO: VALID CRASH
Driver Airbag ON
Passenger Airbag ON
------------------------------
ACC=705 | GYRO=716 | PRESSURE=242 | BABY=NO
SCENARIO: VALID CRASH
Driver Airbag ON
Passenger Airbag ON
```